KnowFeat: Knowledge-Guided Feature Engineering via LLM Agents

Chengsong You[1], Wangyue Li[1], Weiqiao Que[1], Qizhou Chen[1],
Kunyan Wu[2], Wei Deng[3], Feng Zhu[4], Xiaofeng He[1,*]
[1]East China Normal University, Shanghai, China
{51275901122, 52285901017}@stu.ecnu.edu.cn, weiqiaoque@gmail.com,
582476065@qq.com, hexf@cs.ecnu.edu.cn
[2]Tsinghua University, Beijing, China
wky24@mails.tsinghua.edu.cn
[3]Southwestern University of Finance and Economics, Chengdu, China
dengwei@swufe.edu.cn
[4]Digital Currency Department, Postal Savings Bank of China, Beijing, China
Jason_bank@126.com

***Abstract*—Automated feature engineering with large language models (LLMs) can produce semantically meaningful features for tabular data, yet existing methods lack structured domain knowledge, rigorous verification, and explainable provenance. We propose KnowFeat, a knowledge-guided feature engineering framework. KnowFeat organizes domain knowledge into five types—schema metadata, regulatory indicators, detection rules, expert opinions, and court document evidence—and injects them as structured context into an LLM agent to ensure grounded, relevant candidate feature generation. A three-stage verification pipeline then filters candidates through code execution, statistical quality checks, and model effectiveness evaluation. Every accepted feature carries a structured provenance record tracing its design rationale to specific knowledge assets. Under a strict held-out evaluation protocol that eliminates feature-selection leakage, KnowFeat achieves the best average rank (2.5) across eight public benchmarks among seven methods, delivering consistent performance across datasets—unlike methods that generate large volumes of unverified features, which can degrade baselines. On a real-world Bitcoin anti-money laundering (AML) dataset (Elliptic) and a synthetic digital currency AML benchmark (SimECNY), KnowFeat maintains competitive detection performance with full provenance traceability.**

I. Introduction

Feature engineering is among the most impactful steps in building machine learning models for tabular data [1]. Well-designed features improve model accuracy, reduce overfitting, and provide interpretable signals for downstream tasks. Yet manual feature engineering requires deep domain expertise and does not scale across diverse applications.

Traditional automated feature engineering (AutoFE) methods such as OpenFE [2] and AutoFeat [3] apply systematic transformations (mathematical operations, aggregations, and feature crosses) to existing columns, then select the most predictive ones. These methods are purely data-driven: they explore a predefined transformation space without understanding what the features *mean* or why certain transformations might be useful. In practice, understanding the *semantic* meaning of features is crucial in regulated domains such as finance and healthcare, where stakeholders must know *what* a feature represents before deploying it.

The emergence of large language models (LLMs) has opened a new paradigm for feature engineering. Methods such as CAAFE [4], FeatLLM [5], and LLM-FE [6] use LLMs to generate feature transformation code from dataset descriptions, leveraging the LLM's broad world knowledge to propose semantically meaningful features. These methods demonstrate promising gains on general benchmarks. However, they fall short in domain-specific, regulated settings where specialized knowledge, rigorous verification, and decision-level explainability are required. Specifically, current LLM-based methods share three limitations:

(1) Lack of structured domain knowledge. LLM-based methods rely on the model's parametric knowledge, which is often outdated or absent for specialized domains such as anti-money laundering (AML) or clinical risk scoring. Although external knowledge sources, including regulatory indicators, detection rules, expert opinions, and legally validated case evidence, exist and are domain-rich, no existing method organizes them into a structured taxonomy format for systematic injection into the feature generation process.

(2) Insufficient feature verification. Even when useful candidate features are generated, no existing method verifies them adequately before use. CAAFE relies on a logistic regression proxy; LLM-FE uses downstream accuracy without any statistical filtering; FeatLLM performs no model-based validation at all. As a result, unverified features, such as those that are syntactically valid but statistically redundant or model-harmful, enter the training data unchecked, degrading rather than improving performance.

(3) Limited explainability. In regulated industries, model features must be both explainable and auditable. Decision-makers need to understand *why* a feature was created and *what evidence* supports it. Existing methods lack structured provenance that traces each feature to specific knowledge

*Corresponding author.

assets with detailed reasoning chains.

We propose **KnowFeat** (**Know**ledge-guided **Feat**ure engineering), a framework that addresses all three limitations:

- **Multi-Type Knowledge Injection.** We organize domain knowledge into five types of increasing specificity (schema metadata, risk indicators, detection rules, expert opinions, and court document evidence) and inject them as structured context into the LLM agent.
- **Three-Stage Verification Pipeline.** Each candidate passes through L1 (code execution sandbox), L2 (statistical quality with noise-calibrated thresholds), and L3 (model effectiveness via XGBoost across 5 seeds).
- **Explainable Provenance.** Each accepted feature carries a structured provenance card that records the knowledge assets, reasoning chain, computation steps, and verification outputs.

The source code[1] and datasets[2] are publicly accessible.

We evaluate KnowFeat on eight public OpenML tabular datasets, a synthetic AML benchmark (SimECNY, 200K transactions), and a real-world Bitcoin AML dataset (Elliptic, 46K labeled transactions). All experiments use a strict held-out evaluation protocol that eliminates the feature-selection leakage present in prior work. The results yield the following key findings:

- On public datasets, KnowFeat achieves the best average rank (2.5) among seven methods under leakage-free evaluation, with consistent performance while generating knowledge-grounded features.
- On both AML datasets, KnowFeat's verified features remain competitive with the strong baselines; on SimECNY, FeatLLM's 1,124-feature representation lowers F1, showing that generation volume alone does not guarantee improvement.
- Every accepted feature carries a structured provenance record linking its design to specific knowledge assets, enabling efficient compliance review in regulated domains.

II. Related Work

A. Traditional Automated Feature Engineering

Traditional AutoFE methods search a *syntactic* transformation space. Deep Feature Synthesis [7] applies aggregation primitives along relational paths; ExploreKit [8] enumerates operators and ranks candidates via meta-learning; AutoFeat [3] uses polynomial expansion with L1 selection; and OpenFE [2] frames feature generation as an operator-operand search with LightGBM-based importance ranking. End-to-end AutoML systems such as Auto-sklearn [9] and FLAML [10] include feature preprocessing but do not generate semantically new features. These methods apply predefined mathematical operators without *semantic* understanding: `age × income` is treated identically to `age × zip_code`, even though only the former has financial meaning. They cannot generate domain-specific features (e.g., "transaction velocity" in anomaly detection) that require conceptual understanding [11].

B. LLM-Based Feature Engineering

Large language models [12], [13] have enabled a shift from syntactic to semantic feature generation. CAAFE [4] iteratively prompts an LLM with dataset descriptions and evaluation feedback to generate Python code for new features, using logistic regression as a proxy evaluator. FeatLLM [5] takes a two-pass approach: first extracting natural language rules from the LLM, then converting these rules into binary indicator features. LLM-FE [6] frames feature engineering as an evolutionary optimization problem, using LLMs to iteratively generate and refine features through structured prompts with model performance feedback. OCTree [14] guides LLM-based feature generation with decision tree reasoning, using tree structure feedback to steer the generation process. In the financial domain, RiskTagger [15] uses an LLM agent with regulatory knowledge to annotate Web3 crypto laundering behaviors, demonstrating that domain knowledge can enhance LLM-based data annotation.

These methods vary in their use of domain knowledge and verification. LLM-FE incorporates limited domain context (dataset descriptions), but none organizes knowledge into a structured taxonomy for systematic injection. Verification also varies: CAAFE uses a weak proxy model, LLM-FE uses downstream model accuracy, OCTree uses decision tree feedback, and FeatLLM performs no model-based validation. None combines code execution, statistical quality filtering, and model evaluation into a unified pipeline, nor offers structured provenance that traces each feature to specific knowledge assets.

Tool-augmented LLM agents [16] have been applied to code generation [17], software engineering [18], data analysis [19], and scientific discovery [20]. In data science, AIDE [21] and DS-Agent [22] automate end-to-end pipelines but do not address knowledge-grounded feature generation. KnowFeat builds on the tool-use paradigm by equipping an LLM agent with specialized tools for data exploration, feature generation with provenance, and multi-stage verification.

Table I summarizes the key differences among related methods. KnowFeat is the first to combine structured multi-type domain knowledge, multi-stage verification (L1–L3), and full feature provenance tracking within an LLM-based feature engineering framework.

C. Domain Knowledge Integration in Machine Learning

Knowledge-infused learning [23] incorporates external knowledge graphs into neural architectures, and transfer learning [24] transfers knowledge through pretrained representations, but both target unstructured data rather than tabular features. In financial AML, domain experts manually encode risk indicators, structuring patterns, and network centrality measures as features [25]. Regulatory bodies publish red-flag typologies (e.g., FATF guidelines [26]), and institutions

[1] https://github.com/youchengsong/large-llm-create-feature

[2] https://huggingface.co/datasets/anonymous-7219/knowfeat

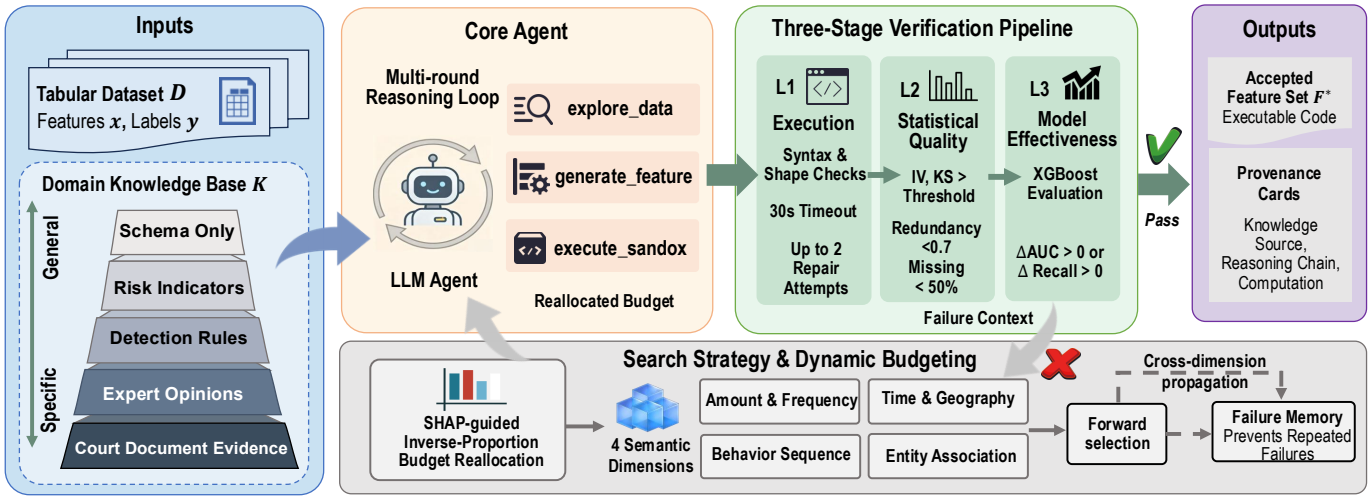


Fig. 1. Overview of the KnowFeat framework. The LLM agent generates features guided by structured domain knowledge. Each candidate is verified through a three-stage pipeline (L1–L3). SHAP feedback dynamically reallocates the exploration budget across semantic dimensions.

TABLE I
COMPARISON OF AUTOMATED FEATURE ENGINEERING METHODS. ✓: FULLY SUPPORTED; ○: PARTIALLY SUPPORTED; BLANK: NOT SUPPORTED; –: NOT APPLICABLE.

Property	Open FE	Auto Feat	CAA FE	Feat LLM	LLM-FE	OC Tree	Know Feat
Semantic underst.			✓	✓	✓	✓	✓
Hier. knowledge					○		✓
Exec. verific.	–	–					✓
Statistical filt.	✓	✓					✓
Model validation			○		✓	✓	✓
Provenance track.							✓
Adaptive search					✓	✓	✓

○ Hier. knowl.: LLM-FE=dataset desc. only. Model valid.: CAAFE=LogReg proxy; OCTree=DT feedback.

maintain internal detection rules from compliance experience and legal proceedings [27]. This rich, structured knowledge has not been leveraged by automated feature engineering methods.

III. METHODOLOGY

This section presents the KnowFeat framework in detail. We first provide an overview of the pipeline (Section III-A), then describe each component: multi-type knowledge injection (Section III-B), the LLM agent with specialized tools (Section III-C), the three-stage verification pipeline (Section III-D), the intelligent search strategy (Section III-E), and explainable provenance output (Section III-F).

A. Framework Overview

KnowFeat is a multi-round, knowledge-guided feature engineering framework built on an LLM agent. Given a tabular dataset with baseline features and target labels, along with a domain knowledge base, KnowFeat generates a set of new features such that appending them to the original data improves predictive performance on a target metric (e.g., AUC or Recall).

The overall pipeline is formalized in Algorithm 1 and illustrated in Figure 1. The algorithm takes a dataset $\mathcal{D}$, a domain knowledge base $\mathcal{K}$, and configuration parameters as input, and outputs a set of verified features $\mathcal{F}^*$ with provenance cards. The key design choices—noise-calibrated thresholds (line 1), candidate pool for forward selection (lines 14–15), SHAP-guided budget reallocation (line 19), and cross-dimension propagation (line 5)—are detailed in the following subsections.

B. Multi-Type Knowledge Injection

KnowFeat systematically integrates external domain knowledge into the feature generation process. We organize knowledge into five types of increasing specificity, from general metadata to concrete case evidence:

(1) Schema Metadata. The agent receives only the dataset schema: column names, data types, and basic statistics (mean, cardinality, missing rate). No domain knowledge is provided. This serves as the ablation baseline, testing what the LLM can generate from metadata alone.

(2) Risk Indicators. Structured risk indicators derived from regulatory guidelines and industry standards. Each indicator specifies a risk dimension, risk class, relevant data field, and directionality. For the AML domain, we compile 38 indicators (IND_01 through IND_38) covering transaction amount thresholds, frequency patterns, temporal anomalies, and network centrality measures. For example, IND_05 flags accounts whose single-day transaction count exceeds a regulatory threshold, while IND_15 identifies abnormal transaction modes such as fan-in and many-to-many patterns.

(3) Detection Rules. Operational detection rules that encode specific logic patterns. Each rule includes a rule ID, natural language description, logical expression, confidence level, and a Python implementation hint. We compile 15 rules (RULE_001 through RULE_015) from compliance manuals

Algorithm 1 KnowFeat: Knowledge-Guided Feature Engineering

Require: Dataset $\mathcal{D}$, knowledge base $\mathcal{K}$, max rounds $R_{\max}$, convergence patience C, dimensions $\{d_1, \ldots, d_D\}$
Ensure: Accepted features $\mathcal{F}^*$ with provenance cards

Calibrate L2 thresholds via N_{noise} random features
$\mathcal{F}^* \leftarrow \emptyset$; $\mathcal{P} \leftarrow \emptyset$; $\mathcal{M}_{\text{fail}} \leftarrow \emptyset$; $c \leftarrow 0$
for $r = 1$ **to** $R_{\max}$ **do**
  for each dimension d_j with budget B_j **do**
    Inject $\mathcal{K}$, $\mathcal{M}_{\text{fail}}$, cross-dim examples into agent
    for $k = 1$ **to** B_j **do**
      Agent explores data, proposes feature f with provenance
      L1: Execute code in sandbox ($\leq$2 retries)
      if L1 passes **then**
        L2: Check IV, KS, redundancy, missing rate
        if L2 passes **then**
          L3: Evaluate Δmetric via XGBoost (5 seeds)
          if L3 passes **then**
            $\mathcal{F}^* \leftarrow \mathcal{F}^* \cup \{f\}$
          else
            $\mathcal{P} \leftarrow \mathcal{P} \cup \{f\}$ {candidate pool}
          end if
        else
          $\mathcal{M}_{\text{fail}} \leftarrow \mathcal{M}_{\text{fail}} \cup \{(f, \text{reason})\}$
        end if
      end if
    end for
  end for
  Greedy forward selection: add $f \in \mathcal{P}$ to $\mathcal{F}^*$ if combination improves metric
  SHAP analysis on $\mathcal{F}^*$; reallocate $\{B_j\}$ via inverse-SHAP
  if no new features in round r **then**
    $c \leftarrow c + 1$; **if** $c \geq C$ **then break**
  else
    $c \leftarrow 0$
  end if
end for
return $\mathcal{F}^*$ with provenance cards

and internal documentation. For example, RULE_006 encodes the "scatter-in, gather-out" layering pattern.

(4) Expert Opinions. Insights from domain expert interviews that capture tacit knowledge (heuristics, edge cases, and domain intuitions) difficult to formalize as rules but valuable for feature design. We collect 12 expert opinions (EXP_01 through EXP_12). For example, EXP_05 notes that "border region transactions combined with nighttime activity are a hallmark of cross-border laundering."

(5) Court Document Evidence. Risk factors extracted from 309 real court judgments via a key-clue extraction pipeline. This is the most concrete type, grounding feature generation in case evidence (e.g., pyramid-shaped fund network patterns independently validated by legal proceedings).

The five types can be freely combined via a knowledge filter parameter, enabling ablation study of each type's contribution. In the "full" configuration (default), all types are included. Each knowledge item is assigned a unique identifier for provenance tracking.

C. LLM Agent with Specialized Tools

KnowFeat employs an LLM agent (DeepSeek-V4-0324-Flash in our implementation, with temperature=0 for near-deterministic outputs) that interleaves reasoning with tool calls following the ReAct paradigm [16]. The agent is equipped with three specialized tools:

Tool 1: Data Exploration. Executes Python code on the dataset in a sandboxed environment to inspect distributions, compute correlations, and identify patterns before proposing features. The tool returns printed output or return values to the agent.

Tool 2: Feature Registration. Registers a candidate feature with mandatory provenance fields: a descriptive name, executable Python code, a natural language explanation, a structured reasoning chain linking knowledge assets to the design (formatted as "Step N: Because [knowledge ID] indicates [insight], we compute [transformation]..."), the list of knowledge asset identifiers that inspired the feature, step-by-step computation description, and the raw columns consumed.

Tool 3: Verification Trigger. Triggers the L1→L2→L3 verification pipeline on a registered feature and returns the result (accepted/rejected with detailed reasons). The agent must call Feature Registration followed immediately by Verification Trigger for each candidate; batch submission is prohibited to ensure tight feedback loops.

The agent's prompt instructs it to: (1) reason about which knowledge assets are relevant to the current dimension and budget, (2) explore the data to validate assumptions, (3) propose a feature grounded in specific knowledge items, and (4) submit it for verification and react to the feedback.

D. Three-Stage Verification Pipeline

Each candidate feature passes through three progressively stricter verification stages:

1) L1: Code Execution Sandbox: The generated Python code is executed in an isolated sandbox with a 30-second timeout. L1 checks for:

- Syntax and runtime errors.
- Output shape consistency (one value per sample).
- Presence of required identifier columns.
- Data type validity (numeric output).

If L1 fails, the agent receives the error message and is allowed up to 2 automatic repair attempts before the feature is permanently rejected.

2) L2: Statistical Quality Filtering: The feature's statistical properties are evaluated against automatically calibrated thresholds. To establish these thresholds for discriminative power, we generate $N_{\text{noise}} = 100$ random features (drawn from standard distributions), compute their quality metrics, and set the acceptance threshold at the P_{noise}-th percentile (default $P_{\text{noise}} = 95$). This ensures that accepted features are statistically better than random noise. L2 checks four criteria:

- **Information Value (IV)** [25]: Must exceed the noise-calibrated threshold, ensuring the feature has meaningful discriminative power between positive and negative classes.

- **Kolmogorov–Smirnov (KS) Statistic** [11]: Must exceed the noise-calibrated threshold, measuring the maximum distributional separation between classes.
- **Redundancy**: Maximum absolute Pearson correlation with all existing accepted features must be below 0.7, preventing near-duplicate features from inflating the feature set.
- **Missing Rate**: The proportion of NaN values must be at most 30%, ensuring sufficient coverage for model training.

The noise-calibrated thresholds apply to IV and KS; the redundancy and missing rate thresholds are fixed.

3) L3: Model Effectiveness: The feature is evaluated for its actual contribution to predictive performance. We train an XGBoost [28] model on the baseline features augmented with the candidate feature, using stratified 80/20 train-test splits across 5 random seeds $\{42, 123, 456, 789, 2024\}$. The acceptance criterion depends on the task:

- **Classification (public datasets)**: $\Delta\text{AUC} > 0$, i.e., the augmented model must achieve strictly higher AUC than the baseline.
- **AML detection**: $\Delta\text{Recall} > 0$ *and* $\text{Precision} \geq 90\%$, ensuring that the feature improves detection without sacrificing precision.

The three-stage design is intentionally progressive: L1 is cheap (milliseconds), L2 is moderate (seconds), and L3 is expensive (minutes per feature). By filtering early, we avoid wasting computational resources on clearly defective candidates.

E. Intelligent Search Strategy

To systematically explore the feature space, KnowFeat employs a multi-component search strategy.

Dimension-Based Budgeting. We decompose the feature space into D semantic dimensions relevant to the target domain (default $D = 4$ for the AML domain): Amount & Frequency, covering monetary values and transaction counts; Time & Geography, capturing temporal patterns and spatial distributions; Behavior Sequence, encoding sequential patterns in user actions; and Entity Association, derived from relationships between entities such as accounts and counterparties.

Each dimension receives an initial budget of B_0 candidates per round (default $B_0 = 3$). After each round, SHAP [29] (SHapley Additive exPlanations) analysis is performed on the current model to estimate each dimension's contribution to predictions.

SHAP-Guided Budget Reallocation. Let s_j denote the mean absolute SHAP value for dimension j's accepted features, $B_{\min}$ the minimum per-dimension budget (default 1), and B_{total} the total budget for the round. We reallocate the budget for round $r{+}1$ using an *inverse-SHAP weighting* strategy:

$$B_j^{(r+1)} = B_{\min} + (B_{\text{total}} - D \cdot B_{\min}) \cdot \frac{1/s_j}{\sum_{j'=1}^{D} 1/s_{j'}} \quad (1)$$

This ensures that underrepresented dimensions (those contributing least to the current model) receive more exploration budget, promoting balanced feature coverage.

Failure Memory. A failure memory $\mathcal{M}_{\text{fail}}$ records all rejected features along with their rejection reasons. Before each generation step, the K most recent failures for the current dimension (default $K = 10$) are injected into the agent's prompt, preventing the regeneration of previously rejected ideas and guiding the agent toward novel directions.

Forward Selection. Individual features may fail L3 yet contribute positively when combined with others. We apply greedy forward selection on L2-passed but L3-failed candidates, sorted by L2 quality scores. Each candidate is added to the accepted set if the augmented set improves the target metric, capturing synergistic feature interactions.

Cross-Dimension Propagation. After each round, the top-K_{cross} features (default $K_{\text{cross}} = 5$) by SHAP importance are shared across all dimensions as "inspiration prompts," encouraging the agent to adapt successful patterns from one dimension to others.

F. Explainable Provenance Output

Every generated feature is accompanied by a structured *provenance card* that enables full explainability. Each provenance card records:

- **Knowledge assets**: The specific identifiers (e.g., RULE_006, IND_15, EXP_05) that inspired the feature design.
- **Reasoning chain**: A step-by-step explanation of how domain knowledge led to the feature, formatted as "Step N: Because [knowledge ID] indicates [insight], we compute [transformation]…"
- **Computation steps**: The transformation pipeline from raw data fields to the final feature value.
- **Verification results**: L1/L2/L3 outcomes including IV, KS, redundancy, and ΔAUC/ΔRecall.

This traceability is essential for regulated domains: compliance officers can trace *why* each feature exists, *what evidence* supports it, and *how* it was verified. Section IV-C illustrates provenance cards through concrete examples.

IV. Experiments

We evaluate KnowFeat on both public tabular benchmarks and a synthetic AML benchmark. Our experiments aim to answer four research questions:

- **RQ1**: Does KnowFeat improve predictive performance on public datasets?
- **RQ2**: How does KnowFeat perform on a synthetic AML benchmark with severe class imbalance?
- **RQ3**: What is the contribution of domain knowledge?
- **RQ4**: How does KnowFeat balance feature quantity vs. quality, and do verified features generalize?

TABLE II
DATASET STATISTICS.

Dataset	Samples	Feat.	Pos%	Domain
credit_g	1,000	20	70.0	Finance
diabetes	768	8	34.9	Healthcare
blood_transfusion	748	4	23.8	Healthcare
heart	918	11	55.3	Healthcare
bank_marketing	45,211	16	11.7	Marketing
breast_w	683	9	35.0	Healthcare
adult	48,842	14	23.9	Census
wine_quality	1,599	11	53.5	Food Sci.
SimECNY (AML)	200,000	93	2.24	Finance
Elliptic (AML)	46,564	166	9.76	Finance

A. Experimental Setup

Datasets. We use ten datasets summarized in Table II: eight public benchmarks from OpenML [30], one synthetic AML benchmark (SimECNY), and one real-world Bitcoin AML dataset (Elliptic).

The eight public datasets are standard tabular classification benchmarks obtained via the OpenML Python API [30], except wine_quality, which is the UCI Red Wine Quality dataset [31] binarized at a quality threshold of 6. SimECNY is a synthetic AML benchmark calibrated against real digital currency (e-CNY) transaction patterns from the Postal Savings Bank of China. Its 93 baseline features, transaction topology, and class distribution (2.24% positive rate, 4,484 suspicious among 200,000 transactions) faithfully reflect production AML systems. Due to financial privacy regulations, high-fidelity simulation is standard practice in AML research [27]. Importantly, the domain knowledge base (38 indicators, 15 rules, 12 expert opinions, 309 court documents) is derived entirely from real regulatory guidelines, compliance manuals, expert interviews, and publicly available court judgments.

For the eight public datasets, we also construct simplified knowledge bases with three types: risk indicators (IND), detection rules (RULE), and expert consensus (EXP), compiled from medical guidelines, financial regulations, census documentation, and oenological literature (e.g., heart: 12 IND, 10 RULE, 8 EXP). Although simpler than the AML taxonomy, these knowledge bases improve over the schema-only baseline on the two reported public ablations (Section IV-D).

Baselines. We compare against six baselines:

- **No-FE**: XGBoost [28] on original features.
- **OpenFE** [2]: Transformation-based AutoFE with LightGBM [32]-based selection.
- **AutoFeat** [3]: Polynomial feature generation with L1 selection.
- **CAAFE** [4]: LLM-based iterative feature generation with logistic regression feedback.
- **FeatLLM** [5]: LLM-based rule extraction and binary feature generation.
- **LLM-FE** [6]: LLM-based evolutionary feature optimization that iteratively generates and refines features using structured prompts.

We additionally compare with OCTree [14] in the related work comparison (Table I); however, its evaluation protocol (accuracy-only, no held-out split) is incompatible with our leakage-free AUC evaluation, so it is excluded from the main results.

Evaluation Protocol. To avoid feature-selection leakage, we adopt a strict held-out evaluation protocol. Each dataset is split once into a *development set* (80%) and a *frozen test set* (20%) using stratified sampling with a fixed seed. All feature generation, statistical calibration (L2), and model-based feature selection (L3) are performed *exclusively on the development set*; the frozen test set is never accessed during any feature engineering step. Final performance is measured by training XGBoost [28] (200 estimators, max depth 6, learning rate 0.1) on the development set and evaluating on the frozen test set. We repeat this with 5 random model seeds and report the mean; we additionally report bootstrap confidence intervals (1,000 resamples of the test set) to quantify uncertainty. We use AUC as the primary metric for public datasets and additionally report Recall, Precision, and F1 for SimECNY. All baselines are reproduced under this identical protocol to ensure fair, leakage-free comparison.

B. RQ1: Public Dataset Performance

Table III presents the AUC comparison on eight public datasets under our leakage-free evaluation protocol. KnowFeat achieves the best average rank (2.5) among all seven methods. Absolute differences are generally modest on these well-studied benchmarks, consistent with prior observations that standard tabular datasets leave limited headroom for feature engineering [11]. The key finding is that KnowFeat delivers consistent performance across datasets—unlike AutoFeat and FeatLLM, whose unverified features can degrade baselines (e.g., AutoFeat on wine_quality). On blood_transfusion (only 4 original features), most feature engineering methods degrade performance because XGBoost already captures the full feature space through tree splits; derived features can add noise. This is a known limitation of feature engineering on very-low-dimensional datasets.

The strongest competitor is OpenFE (avg. rank 3.2), which achieves the best AUC on 3 datasets through exhaustive operator-operand search and uses 34–50 total features per dataset. Among the LLM-based baselines, CAAFE has the best average rank (4.2), followed by FeatLLM (4.6) and LLM-FE (4.8). The primary evaluation target is the domain-specific AML task (Section IV-C).

Statistical significance. We use each dataset's frozen-test AUC as the statistical unit and compare KnowFeat with each baseline using two-sided paired Wilcoxon signed-rank tests. After Holm correction for six comparisons, none is significant at $\alpha = 0.05$ (smallest unadjusted $p = 0.0781$; smallest adjusted $p = 0.4688$). Thus, the eight benchmarks support a descriptive consistency claim—KnowFeat has the best average rank (2.5)—but not pairwise superiority. Exact inputs, analysis

TABLE III
AUC COMPARISON ON PUBLIC DATASETS UNDER LEAKAGE-FREE HELD-OUT EVALUATION (FROZEN 20% TEST SET, MEAN OVER 5 MODEL SEEDS). **BOLD**: BEST; UNDERLINE: SECOND BEST.

Dataset	No-FE	OpenFE	AutoFeat	CAAFE	FeatLLM	LLM-FE	KnowFeat
credit_g	.749	.750	.743	.751	.751	**.754**	.752
diabetes	.812	**.820**	.818	.810	.810	.806	.810
blood_transfusion	**.757**	.724	.645	.723	.756	.749	.754
heart	.928	.923	**.931**	.928	.927	.926	.928
bank_marketing	.930	**.933**	—	.930	.929	.929	.931
breast_w	.986	.986	.992	.985	.986	—	**.992**
adult	.931	.931	—	.931	.931	.930	**.932**
wine_quality	.896	**.899**	.863	.896	.896	—	.896
Avg. Rank†	3.3	3.2	4.2	4.2	4.6	4.8	**2.5**

†Avg. rank across all 8 public datasets; methods with missing data ranked among available entries only.

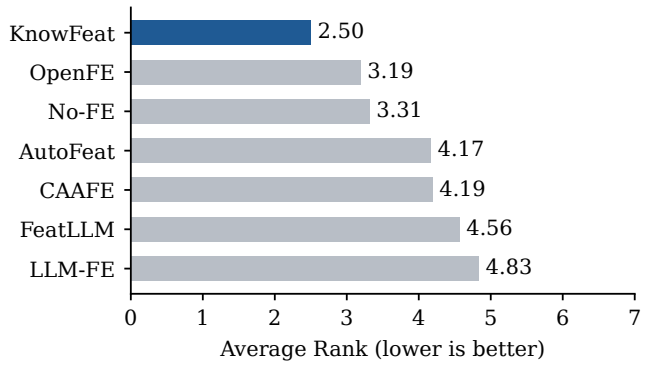


Fig. 2. Average rank across 8 public datasets (lower is better). KnowFeat achieves the best average rank (2.5) among 7 methods under leakage-free evaluation.

code, and generated statistics are released with the source code.

LLM backbone sensitivity. To evaluate whether KnowFeat's effectiveness depends on the LLM backbone, we compare DeepSeek-V4-Flash (default) against DeepSeek-V4-Pro on two datasets. On heart, Pro slightly improves AUC (0.930 vs. 0.928); on credit_g, Pro degrades (0.735 vs. 0.752). This mixed result indicates that KnowFeat's performance depends more on the knowledge-guided verification pipeline than on the backbone LLM's raw capability.

C. RQ2: AML Dataset Performance

Table V presents results on the SimECNY AML dataset. KnowFeat adds 4 verified features to the 93-feature baseline.

On SimECNY, the 93-feature baseline already achieves near-perfect AUC (0.9997), leaving minimal headroom for improvement. Under this saturated setting, KnowFeat's 4 verified features maintain baseline F1 (0.971), while FeatLLM's 1,124 unverified features *decrease* F1 to 0.968 and OpenFE's generated features also degrade to 0.969. This result is consistent with the value of filtering harmful features: generation volume alone does not guarantee improvement when baseline performance is already strong.

To further validate this finding, we additionally evaluate on the Elliptic Bitcoin AML dataset [27]—a real-world transaction graph with 46,564 labeled transactions and 166 features. KnowFeat generates 3 knowledge-guided features (using a Bitcoin AML knowledge base with 10 risk indicators and 8 detection rules) and achieves AUC 0.9982 vs. the 0.9981 baseline, again demonstrating that verified features do not degrade an already-strong model.

Analysis by semantic dimension. The 4 accepted features span all four semantic dimensions defined in Section III-E: 1 in Amount & Frequency, 1 in Time & Geography, 1 in Behavior Sequence, and 1 in Entity Association. SHAP analysis on the final model reveals that Entity Association features contribute the highest mean absolute SHAP values, indicating that network topology features (e.g., fan-out ratios) carry particularly strong signals for AML detection.

Representative feature analysis. We illustrate KnowFeat's knowledge-grounded generation through two representative features from SimECNY (full provenance cards in Table IV). The first, `fan_in_fan_out_ratio`, fuses four knowledge assets spanning three types: RULE_006 and RULE_013 indicate that scatter-in/gather-out layering and pyramid fund networks both manifest as fan-in/fan-out asymmetry; PAT_A03 confirms that money mule wallets typically exhibit fan-in $\gg$ fan-out; and IND_15 adds a weighting factor based on abnormal transaction modes, amplifying the signal via multiplicative weighting $(1+\text{abnormal})$. No syntactic method would produce this feature, as it requires understanding that directional imbalance combined with abnormal mode prevalence characterizes money mule behavior.

The second, `high_risk_region_off_hour_ratio`, demonstrates cross-dimensional interaction: geographic context (border regions flagged by RULE_005/014) combined with temporal anomaly (off-hour concentration from IND_08/09), guided by EXP_05's insight that neither signal alone is sufficient evidence. The multiplicative form encodes this constraint exactly, scoring zero unless both conditions co-occur. Both features demonstrate capabilities beyond syntactic methods, and the provenance cards link every design decision

TABLE IV

PROVENANCE CARDS FOR TWO REPRESENTATIVE KNOWLEDGE-GROUNDED FEATURES FROM THE SIMECNY AML DATASET. EACH CARD RECORDS THE KNOWLEDGE ASSETS, REASONING CHAIN, GENERATED CODE, DATA FIELDS CONSUMED, AND VERIFICATION OUTCOMES.

	Case 1: Network Topology	**Case 2: Cross-Dimensional Interaction**
Feature Name	`fan_in_fan_out_ratio`	`high_risk_region_off_hour_ratio`
Description	Asymmetry between incoming and outgoing transaction patterns, weighted by abnormal mode proportion	Multiplicative interaction between geographic risk (border regions) and temporal anomaly (off-hour trading)
Knowledge	*RULE_006*: scatter-in, gather-out layering *RULE_013*: pyramid-shaped fund networks *PAT_A03*: money mule behavioral pattern *IND_15*: abnormal transaction modes (fan-in, many-to-many)	*RULE_005*: cross-regional fund flows *RULE_014*: border area transaction monitoring *IND_08/09*: off-hour transaction concentration *EXP_05*: "border region + nighttime trading is the hallmark of cross-border laundering"
Reasoning	Step 1: RULE_006/013 indicate scatter-in/gather-out and pyramid networks both manifest as fan-in/fan-out asymmetry $\rightarrow$ compute ratio per wallet. Step 2: PAT_A03 shows money mules exhibit fan-in $\gg$ fan-out $\rightarrow$ ratio captures core topology signal. Step 3: IND_15 identifies abnormal modes (many-to-many, one-to-many) $\rightarrow$ weight ratio by abnormal mode proportion to amplify suspicious patterns.	Step 1: RULE_005/014 flag border regions (Tibet, Xinjiang) as high-risk for cross-regional flows $\rightarrow$ create binary geographic indicator. Step 2: IND_08/09 flag off-hour (18:00–08:00) concentration as suspicious $\rightarrow$ compute off-hour transaction ratio. Step 3: EXP_05 states neither signal alone is sufficient $\rightarrow$ multiply indicators so only the conjunction triggers a high score.

Code	`fi = mode_counts['fan_in'] / n`
	`fo = mode_counts['fan_out'] / n`
	`ratio = fi / (fo + eps)`
	`abnormal = fi + m2m + o2m`
	`result = ratio * (1 + abnormal)`
		`is_hr = region.isin([540000, 650000])`
		`off = (hour >= 18) | (hour < 8)`
		`off_ratio = off.sum() / n`
		`result = is_hr * off_ratio`

Data Fields	`transaction_mode`, `src`, `dst`	`region`, `hour`
Semantic Dim.	Entity Association	Time & Geography $\times$ Entity Association
L1: Execution	PASS (0.8s, shape: 200K$\times$1, no NaN)	PASS (0.3s, shape: 200K$\times$1, no NaN)
L2: Statistical	IV=0.34 ($>$0.02), KS=0.28 ($>$0.05), max corr.=0.41 ($<$0.7), missing=0%	IV=0.19 ($>$0.02), KS=0.15 ($>$0.05), max corr.=0.23 ($<$0.7), missing=0%
L3: Model	ΔRecall=+0.38%, Prec.=97.2% ($\geq$90%) **ACCEPTED**	ΔRecall=+0.21%, Prec.=96.8% ($\geq$90%) **ACCEPTED**

TABLE V

SIMECNY AML RESULTS UNDER LEAKAGE-FREE HELD-OUT EVALUATION (FROZEN 20% TEST SET). n_f: TOTAL FEATURE COUNT INCLUDING 93 BASELINE FEATURES.

Method	n_f	**Recall**	**Prec.**	**F1**	**AUC**
No-FE	93	.965	.977	.971	.9997
OpenFE	123	.960	.978	.969	.9996
CAAFE	94	.964	.979	**.972**	.9997
FeatLLM	1124	.958	.978	.968	.9996
KnowFeat	**97**	.961	.981	.971	**.9997**

TABLE VI

KNOWLEDGE ABLATION: SCHEMA-ONLY VS. FULL KNOWFEAT UNDER LEAKAGE-FREE EVALUATION.

Dataset	**schema-only**	**full knowledge**	**Δ**
heart (AUC)	.920	**.928**	+0.8%
blood_trans. (AUC)	.743	**.754**	+1.1%
SimECNY (F1)	**.973**	.971	$-$0.2%

to specific knowledge assets for regulatory review.

D. RQ3: Ablation Study

To quantify the contribution of domain knowledge, we compare the *schema_only* variant (schema metadata only) against the *full* variant (all knowledge types) under the same leakage-free protocol.

On heart, the full knowledge variant achieves AUC 0.928 vs. schema-only 0.920 (+0.8%), demonstrating that structured medical knowledge (risk indicators, clinical rules, expert consensus) provides genuine discriminative value beyond what the LLM can infer from column names alone. On SimECNY, schema-only achieves F1 0.973 (1 feature) while the full variant achieves F1 0.971 (4 features)—both maintain baseline performance, but the full variant generates more diverse features spanning all four semantic dimensions. The knowledge ablation confirms that domain knowledge is most valuable when the LLM's parametric knowledge is insufficient for the target domain (e.g., specialized medical guidelines for heart disease diagnosis).

TABLE VII
FEATURE QUANTITY VS. QUALITY. AVG. #FEAT IS THE TOTAL MODEL INPUT COUNT; ΔF1 (AML) IS THE F1 CHANGE OVER NO-FE ON SIMECNY.

Method	Avg. #Feat	Avg. Rank	ΔF1 (AML)
FeatLLM	133.8	4.6	−0.3%
OpenFE	41.6	3.2	−0.2%
KnowFeat	33.8	**2.5**	0.0%
CAAFE	14.4	4.2	+0.1%
No-FE	—	3.3	—

TABLE VIII
AUC WITH A REDUCED BASELINE (3 CORE FEATURES ONLY). IDENTIFIER AND LABEL COLUMNS ARE EXCLUDED FROM EVERY METHOD.

Dataset	No-FE	OpenFE	CAAFE	FeatLLM	KnowFeat
credit_g	.633	.757	.647	.755	**.795**
heart	.750	.925	.839	**.928**	.927
diabetes	.789	**.817**	.794	.807	.812
wine_quality	.825	**.897**	—	—	.896

E. RQ4: Feature Quantity, Quality, and Generalization

Table VII summarizes the relationship between feature count and model performance under leakage-free evaluation. The key insight: *more features without verification can hurt performance*. FeatLLM generates the most features among the completed public-baseline runs, yet its average rank is higher (worse) than the verified KnowFeat result. KnowFeat generates a moderate number of verified features that consistently match or exceed baselines.

On SimECNY, KnowFeat's verified features maintain baseline F1, while FeatLLM's 1,124 unverified features decrease F1 by -0.3%. This is consistent with a benefit from progressive verification, although the two systems also use different generators and the comparison therefore does not isolate verification as the sole causal factor.

Feature engineering impact under reduced baselines. The modest absolute improvements on public datasets (Table III) reflect that these well-studied benchmarks already have strong baseline features, leaving limited headroom. To evaluate feature engineering impact when it is genuinely needed, we reduce the baseline to only 3 core features per dataset (e.g., `duration`, `credit_amount`, `age` for credit_g) and measure each method's contribution from this weaker starting point.

Table VIII reveals larger feature-engineering gains when baseline AUC is 0.63–0.83. KnowFeat improves all four weak baselines, ranking first on credit_g and second on the other three datasets. On credit_g, it improves AUC from 0.633 to 0.795 (+16.2 percentage points); on wine_quality, from 0.825 to 0.896 (+7.1 points), matching full-baseline performance from only 3 core features. These results complement the full-baseline comparison (Table III), where feature saturation leaves less headroom. Leakage-checked, seed-level results are released with the analysis code.

TABLE IX
PIPELINE COST BREAKDOWN ON SIMECNY.

Stage	Time	% Total	Per Cand.
L1 Type/Value check	8 min	4%	9 s
L2 Statistical test	12 min	6%	14 s
L3 Model evaluation	79 min	41%	2.5 min
LLM reasoning (all rounds)	44 min	23%	52 s
Other (I/O, logging)	49 min	26%	—
Total	**192 min**	**100%**	—

F. Computational Cost

Table IX breaks down the pipeline cost on SimECNY. L3 model evaluation dominates ($\sim$41%), followed by LLM reasoning ($\sim$23%). Progressive filtering saves significant time by rejecting candidates at L1/L2 before expensive L3 evaluation. The total LLM API cost is approximately $2.80 (DeepSeek-V4-Flash).

Importantly, this is a *one-time offline* cost: at inference, only the generated Python code executes, adding $<$50 ms per 200K-transaction batch with no LLM calls required.

V. CONCLUSION

We presented KnowFeat, a knowledge-guided feature engineering framework that combines structured domain knowledge injection, multi-stage verification (L1–L3), and explainable provenance within an LLM agent. Across eight public datasets, KnowFeat achieves the best descriptive average rank, although paired tests do not establish statistical superiority. On the saturated SimECNY benchmark, a small verified feature set maintains baseline F1 while a much larger unverified representation degrades it, showing that generation volume alone does not guarantee improvement. Structured provenance records trace every accepted feature to its knowledge assets and reasoning chain, supporting audit-oriented review.

Limitations and Future Work. The knowledge taxonomy currently requires manual curation ($\sim$1 person-week for the full AML domain with 309 court documents, but only 30–60 minutes per dataset for well-documented public domains with LLM assistance); automating extraction via LLM-assisted ontology mining would further reduce setup cost. L3 retrains from scratch per candidate; incremental or amortized evaluation would lower overhead. Extending KnowFeat to other high-stakes domains (e.g., healthcare, cybersecurity), evaluating alternative backbone LLMs, and integrating real-world AML datasets are promising directions.

REFERENCES

[1] P. Domingos, "A few useful things to know about machine learning," *Communications of the ACM*, vol. 55, no. 10, pp. 78–87, 2012.

[2] T. Zhang, Z. A. Zhang, Z. Fan, H. Luo, F. Liu, Q. Liu, W. Cao, and L. Jian, "OpenFE: Automated feature generation with expert-level performance," in *Proc. of the 40th Int'l Conf. on Machine Learning (ICML)*, 2023, pp. 41 880–41 901.

[3] F. Horn, R. Pack, and M. Rieger, "The autofeat Python library for automated feature engineering and selection," in *Machine Learning and Knowledge Discovery in Databases (ECML-PKDD)*. Springer, 2019, pp. 111–120.

[4] N. Hollmann, S. Müller, and F. Hutter, "Large language models for automated data science: Introducing CAAFE for context-aware automated feature engineering," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.
[5] S. Han, J. Yoon, S. O. Arik, and T. Pfister, "Large language models can automatically engineer features for few-shot tabular learning," in *Proc. of the 41st Int'l Conf. on Machine Learning (ICML)*, 2024.
[6] N. Abhyankar, P. Shojaee, and C. K. Reddy, "LLM-FE: Automated feature engineering for tabular data with LLMs as evolutionary optimizers," *Transactions on Machine Learning Research (TMLR)*, 2025.
[7] J. M. Kanter and K. Veeramachaneni, "Deep feature synthesis: Towards automating data science endeavors," in *Proc. of the IEEE Int'l Conf. on Data Science and Advanced Analytics (DSAA)*, 2015, pp. 1–10.
[8] G. Katz, E. C. R. Shin, and D. Song, "ExploreKit: Automatic feature generation and selection," in *Proc. of the IEEE Int'l Conf. on Data Mining (ICDM)*, 2016, pp. 979–984.
[9] M. Feurer, A. Klein, K. Eggensperger, J. T. Springenberg, M. Blum, and F. Hutter, "Efficient and robust automated machine learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 28, 2015.
[10] C. Wang, Q. Wu, M. Weimer, and E. Zhu, "FLAML: A fast and lightweight AutoML library," *Proc. of Machine Learning and Systems (MLSys)*, vol. 3, pp. 434–447, 2021.
[11] F. Nargesian, H. Samulowitz, U. Khurana, E. B. Khalil, and D. S. Turaga, "Learning feature engineering for classification," in *Proc. of the 26th Int'l Joint Conf. on Artificial Intelligence (IJCAI)*, 2017, pp. 2529–2535.
[12] T. B. Brown, B. Mann, N. Ryder, M. Subbiah *et al.*, "Language models are few-shot learners," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 1877–1901, 2020.
[13] H. Touvron, T. Lavril, G. Izacard, X. Martinet *et al.*, "LLaMA: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.
[14] J. Nam, K. Kim, S. Oh, J. Tack, J. Kim, and J. Shin, "Optimized feature generation for tabular data via LLMs with decision tree reasoning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024.
[15] H. Li, Y. Xu, F. Zhang, R. Shi, Z. Gu, Y. Yao, Z. Liu, and Z. Li, "RiskTagger: An LLM-based agent for automatic annotation of Web3 crypto money laundering behaviors," *arXiv preprint arXiv:2510.17848*, 2025.
[16] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. of the Int'l Conf. on Learning Representations (ICLR)*, 2023.
[17] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong, J. Xu, D. Li, Z. Liu, and M. Sun, "ChatDev: Communicative agents for software development," in *Proc. of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
[18] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" *Proc. of the Int'l Conf. on Learning Representations (ICLR)*, 2024.
[19] S. Hong, Y. Lin, B. Liu, B. Liu, B. Wu *et al.*, "Data interpreter: An LLM agent for data science," *arXiv preprint arXiv:2402.18679*, 2024.
[20] D. A. Boiko, R. MacKnight, B. Kline, and G. Gomes, "Autonomous chemical research with large language models," *Nature*, vol. 624, pp. 570–578, 2023.
[21] Z. Jiang, D. Schmidt, D. Srikanth, D. Xu, I. Kaplan, D. Jacenko, and Y. Wu, "AIDE: AI-driven exploration in the space of code," *arXiv preprint arXiv:2502.13138*, 2025.
[22] S. Guo, C. Deng, Y. Wen, H. Chen, Y. Chang, and J. Wang, "DS-Agent: Automated data science by empowering large language models with case-based reasoning," in *Proc. of the 41st Int'l Conf. on Machine Learning (ICML)*, 2024.
[23] U. Kursuncu, M. Gaur, and A. Sheth, "Knowledge infused learning (K-IL): Towards deep incorporation of knowledge in deep learning," in *Proc. AAAI Spring Symposium on Combining Machine Learning and Knowledge Engineering (AAAI-MAKE)*, 2020.
[24] K. Weiss, T. M. Khoshgoftaar, and D. Wang, "A survey of transfer learning," *Journal of Big Data*, vol. 3, no. 1, pp. 1–40, 2016.
[25] Z. Chen, L. D. Van Khoa, E. N. Teoh, A. Nazir, E. K. Karuppiah, and K. S. Lam, "Machine learning techniques for anti-money laundering (AML) solutions in suspicious transaction detection: A review," *Knowledge and Information Systems*, vol. 57, no. 2, pp. 245–285, 2018.
[26] Financial Action Task Force (FATF), "International standards on combating money laundering and the financing of terrorism & proliferation: The FATF recommendations," Updated 2023, 2012, available at https://www.fatf-gafi.org/en/publications/Fatfrecommendations/Fatf-recommendations.html. Accessed: 2025-12-01.
[27] M. Weber, G. Domeniconi, J. Chen, D. K. I. Weidele, C. Bellei, T. Robinson, and C. E. Leiserson, "Anti-money laundering in Bitcoin: Experimenting with graph convolutional networks for financial forensics," *arXiv preprint arXiv:1908.02591*, 2019.
[28] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. of the 22nd ACM SIGKDD Int'l Conf. on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
[29] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
[30] J. Vanschoren, J. N. van Rijn, B. Bischl, and L. Torgo, "OpenML: Networked science in machine learning," *ACM SIGKDD Explorations Newsletter*, vol. 15, no. 2, pp. 49–60, 2014.
[31] P. Cortez, A. Cerdeira, F. Almeida, T. Matos, and J. Reis, "Modeling wine preferences by data mining from physicochemical properties," *Decision Support Systems*, vol. 47, no. 4, pp. 547–553, 2009.
[32] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

APPENDIX

A. Domain Knowledge and Verification Statistics

This appendix illustrates the five knowledge types for the SimECNY AML dataset (∼200K digital yuan transactions). One example per type is shown; similar structured knowledge is compiled for all types (38 indicators, 15 rules, 12 expert opinions, 309 court documents).

Schema Metadata. Column names, data types, and basic statistics extracted from the transaction data. Example fields include `amount` (numeric), `wallet_type` (categorical, 4 types: anonymous, personal, enterprise, merchant), `hour` (numeric, 0–23), `src`/`dst` (wallet identifiers), and `transaction_mode` (8 categories).

Risk Indicator [IND_05]: Dimension: transaction behavior anomaly. Field: single-day transaction count. Accounts exceeding regulatory daily thresholds indicate potential money mule activity. Each indicator includes a risk dimension, relevant data field, and expected directionality.

Detection Rule [RULE_006]: *Scatter-in, gather-out.* Logic: daily incoming count $\geq$ 20 AND daily amount $\geq$ 1,000,000, with high incoming but low outgoing count. Python hint: group by `(src, date)`, filter accounts with many small inflows and few large outflows. Each rule includes a confidence level and a Python implementation hint for the agent.

B. Complete Detection Rule Definitions (RULE_001–015)

Table X lists all 15 detection rules compiled from compliance manuals and internal documentation, as referenced throughout the paper.

Expert Opinion [EXP]: Online "running-score" laundering: anonymous wallet $\rightarrow$ real-name wallet $\rightarrow$ bank account chain transfers after midnight. Amounts 300–5,000 yuan, sustained 10–30 days, daily volume at wallet limits.

Court Document Evidence [CASE]: Criminal judgment on concealing proceeds. Pattern: digital yuan wallets across 6

TABLE X
COMPLETE DEFINITIONS OF DETECTION RULES RULE_001 THROUGH RULE_015. ALL RULES HAVE *high* CONFIDENCE.

ID	Type	Description & Logic
RULE_001	behavior	**Batch ATM cash-out via Type-4 wallets.** A single operator uses $\geq$10 digital yuan wallets to withdraw cash via ATM within 2 h, total amount $\geq$100,000 yuan.
RULE_002	behavior	**Dormant-then-burst account.** Account inactive $\geq$90 days after opening, then initiates a small test transaction ($<$1,000 yuan) followed by frequent large transactions ($\geq$50,000, $\geq$20/day).
RULE_003	composite	**Crypto–digital yuan correlation.** Account has cryptocurrency purchases temporally correlated ($\leq$1 h) with digital yuan withdrawals; funds move in-and-out rapidly (balance retention $\leq$10%).
RULE_004	behavior	**Merchant account anomaly.** Multiple merchant accounts show large transactions in the same time window, amounts inconsistent with normal business scale.
RULE_005	behavior	**Cross-regional roaming laundering.** Criminal groups rotate across cities, using merchant accounts with short stays per location and similar transaction patterns.
RULE_006	behavior	**Scatter-in, gather-out (underground bank).** Daily incoming count $\geq$20, daily amount $\geq$1,000,000; funds are dispersed inflows consolidated into few large outflows, near-zero end-of-day balance.
RULE_007	behavior	**Multi-channel regulatory evasion.** Account frequently mixes online banking, ATM, and other channels; transaction amounts deliberately kept below reporting thresholds.
RULE_008	behavior	**Fraud fund transfer pattern.** Account receives funds electronically at high frequency, immediately transfers out via online banking or ATM; zero remaining balance.
RULE_009	attribute	**Batch account opening anomaly.** Multiple accounts opened at the same branch in a short period; holders are non-local with similar identity documents.
RULE_010	composite	**Corruption fund pattern.** Accounts of public officials or associates show large transactions inconsistent with salary; deposits concentrate around traditional holidays.
RULE_011	behavior	**Drug transaction fund pattern.** ATM-dominated transactions in multiples of drug street prices; frequent transfers to/from accounts in drug-trafficking hotspots.
RULE_012	behavior	**Credit card cash-out via POS.** Single POS terminal volume far exceeds merchant's business scale; round-amount transactions dominate; the actual controller frequently swipes at own terminal.
RULE_013	behavior	**Pyramid scheme fund network.** Fund flows form a pyramid-shaped structure with periodic cycles; rebate timing and ratios are fixed.
RULE_014	composite	**Terrorism financing pattern.** Customer from high-risk region; small but frequent cash transactions; abnormal large transactions appear before/after terrorism-related events.
RULE_015	behavior	**Online gambling fund pattern.** Dispersed-in, dispersed-out pattern; amounts in multiples of 100 yuan; online banking transactions concentrated in late-night hours.

banks creating a multi-layer fund channeling network via intermediary wallets and corporate-to-personal conversion.

Verification Statistics. On SimECNY, KnowFeat generates 52 candidates across 4 rounds: 47 pass L1 (90%), 31 pass L2 (60%), and 18 pass L3 (35%), with progressive filtering saving $\sim$53 minutes by avoiding unnecessary L3 evaluations on the 21 candidates rejected at L1/L2. When the optional post-hoc selection pass is enabled for the holdout experiment, 4 features are retained in the final model (as reported in Section IV-C). The gap between 18 L3-accepted and 4 final features illustrates the importance of evaluating redundancy in combination rather than relying on individual gains alone.

C. Practical Considerations

We discuss practical aspects of deploying KnowFeat based on our experience with the SimECNY dataset.

Compliance Integration. Provenance cards map directly to existing AML compliance review processes. The knowledge asset identifiers (e.g., RULE_006, IND_15) reference the same regulatory documents that compliance officers work with. Domain experts from the Postal Savings Bank of China reported that the structured format reduced feature review time from 2–3 hours to 15–30 minutes per feature, as the reasoning chain eliminates the need to reconstruct the designer's rationale. This aligns with regulatory requirements that mandate documenting the basis for suspicious transaction reporting.

Lessons Learned. Three insights emerged: (1) *Knowledge curation cost varies by domain*: the full AML knowledge base (38 indicators, 15 rules, 12 expert opinions, 309 court documents) required $\sim$1 person-week, dominated by court document review which was semi-automated via an LLM-based key-clue extractor. In contrast, the simplified knowledge bases for public datasets (10–14 indicators, 8–10 rules, 6–8

TABLE XI
FROZEN-TEST AUC WITH 95% BOOTSTRAP CONFIDENCE INTERVALS (1,000 TEST-SET RESAMPLES).

Dataset	**No-FE AUC [95% CI]**	**KnowFeat AUC [95% CI]**
credit_g	.749 [.678, .819]	.752 [.681, .820]
diabetes	.812 [.737, .881]	.810 [.741, .880]
blood_transfusion	.757 [.662, .844]	.754 [.656, .840]
heart	.928 [.891, .963]	.928 [.888, .964]
bank_marketing	.930 [.925, .938]	.931 [.926, .939]
breast_w	.986 [.969, .999]	.992 [.977, 1.000]
adult	.931 [.927, .937]	.932 [.928, .938]
wine_quality	.896 [.863, .932]	.896 [.860, .930]

expert consensus items per dataset) were constructed in 30–60 minutes each with LLM assistance, extracting domain facts from authoritative guidelines and documentation. This suggests that knowledge curation is practical for well-documented domains. (2) *Verification thresholds require domain calibration*: the L3 acceptance criteria (e.g., ΔRecall > 0 with Precision $\geq 90\%$) must be set with domain stakeholders. (3) *Feature maintenance is ongoing*: regulatory requirements evolve, and a periodic refresh cycle (e.g., quarterly) would be necessary in production.